

# Machine Learning Lab

Yukun Jiao

2025-08-20

## Lab 1

### Part 1: Bernie Sanders and Donald Trump tweets.

1.

```
library(data.table)
library(glmnet)
library(ggplot2)
library(mlbench)
library(caret)
library(splines)
library(ggeffects)
```

```
# load data
trumpbernie <- fread(file = "~/Documents/ML/Labs/Lab1/trumpbernie.csv")
dim(trumpbernie)
```

```
[1] 1003 1496
```

The dataset consists of 1003 observations and 1496 variables. Since the number of variables substantially exceeds the number of observations, the dataset is high-dimensional. Under such conditions, a standard logistic regression model is likely to perform poorly for prediction, due to overfitting and multicollinearity.

2.

```
mymodel <- glm(trump_tweet ~ ., data = trumpbernie, family="binomial")
```

Warning: glm.fit: algorithm did not converge

```
sum(!is.na(coef(mymodel)))
```

```
[1] 1001
```

```
coef(mymodel)[(1001:1050)]
```

|           |            |            |          |            |          |           |
|-----------|------------|------------|----------|------------|----------|-----------|
| popul     | popular    | possibl    | post     | potenti    | poverti  | power     |
| 349.33967 | -771.91767 | -30.26994  | 25.75710 | -380.91299 | 59.38732 | NA        |
| practic   | prayer     | prefer     | premium  | prepar     | pres     | prescript |
| NA        | NA         | NA         | NA       | NA         | NA       | NA        |
| present   | presid     | presidenti | press    | pressur    | pretti   | prevent   |
| NA        | NA         | NA         | NA       | NA         | NA       | NA        |
| previous  | price      | primari    | prime    | prioriti   | prison   | privat    |
| NA        | NA         | NA         | NA       | NA         | NA       | NA        |
| privileg  | pro        | problabl   | problem  | process    | proclaim | produc    |
| NA        | NA         | NA         | NA       | NA         | NA       | NA        |
| product   | profit     | program    | progress | project    | promis   | proper    |
| NA        | NA         | NA         | NA       | NA         | NA       | NA        |
| propos    | protect    | protest    | proud    | provid     | public   | puerto    |
| NA        | NA         | NA         | NA       | NA         | NA       | NA        |
| pull      |            |            |          |            |          |           |
| NA        |            |            |          |            |          |           |

There are many coefficients that are missing. In fact, 495 variables were identified as singular, and their coefficients could not be estimated due to perfect collinearity with other predictors.

```
# Extract predictions on training data & observed values
comparison_df <- data.frame(train_predictions=mymodel$fitted.values,
observed=mymodel$y)
# Apply prediction threshold
comparison_df$train_predictions<-ifelse(comparison_df$train_predictions>=0.5,
yes = 1,
no = 0)
```

```
# Compute accuracy (scale: 0-1, 0=0%, 1=100%)
nrow(comparison_df[comparison_df$train_predictions==comparison_df$observed,]) /
nrow(comparison_df)
```

```
[1] 1
```

The training accuracy here is 1 (100%). This indicates that the model is overfitted, and in such cases, the predictive capacity of the model is often poor.

### 3.

```
# format the outcome variable as a factor variable
trumpbernie$trump_tweet <- factor(trumpbernie$trump_tweet, levels = c(0,1))

# Use package "caret" to do k-fold cross-validation
tc <- caret::trainControl(method = 'cv', number = 3)

set.seed(12345) # Set to ensure that folds are the same across models
linear_model <- caret::train(trump_tweet ~ .,
                             data = trumpbernie,
                             method = "glm",
                             trControl = tc,
                             family = "binomial")
```

Some warnings were issued during model fitting, including non-convergence of the algorithm and rank-deficiency in the fitted model.

```
linear_model$results$Accuracy
```

```
[1] 0.533384
```

```
pred_class <- predict(linear_model, newdata = trumpbernie)
train_acc <- mean(pred_class == trumpbernie$trump_tweet)
train_acc
```

```
[1] 1
```

The model achieved a test accuracy of 0.533384 indicating that its predictions are essentially no better than random guessing, which is consistent with my expectations. These results suggest that the model is severely overfitted.

4.

```
X <- trumpbernie[,-c('trump_tweet'),with=F] # Exclude response and id columns.
X <- as.matrix(X) # Make input data into a matrix

cvglmnet <- cv.glmnet(x = X,
                      y = trumpbernie$trump_tweet,
                      nfolds = 5,
                      standardize = TRUE,
                      family='binomial',
                      alpha=0,
                      type.measure = 'class')

cvglmnet$lambda.min
```

```
[1] 3.275879
```

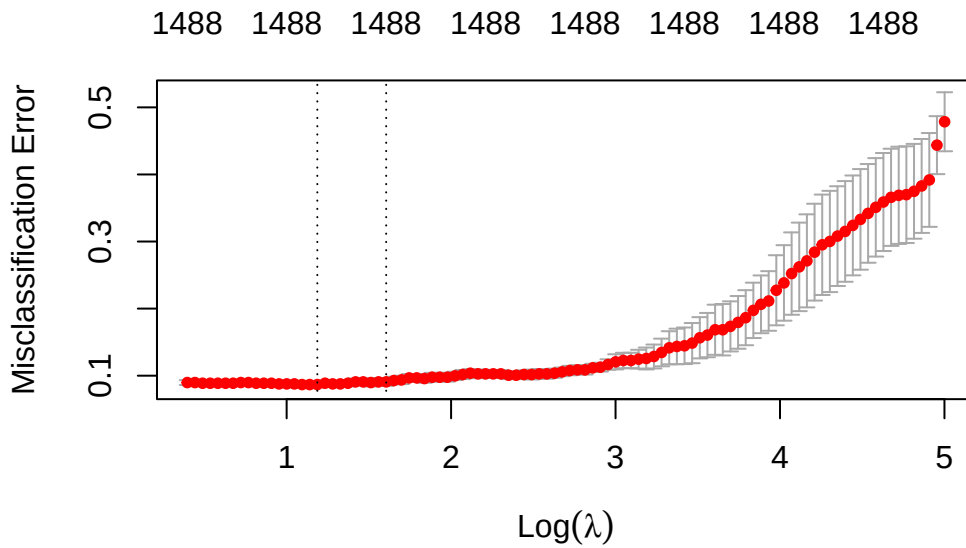
```
cvglmnet$cvm[cvglmnet$index[[1]]]
```

```
[1] 0.08673978
```

The value of  $\lambda$  that minimizes the test error is 3.275879, corresponding to a test accuracy of 0.9132602, indicating that it is a better-performing model. The previous model had higher variance because its greater flexibility caused the results to be highly sensitive, leading to larger variance.

5.

```
plot(cvglmnet, sign.lambda = 1)
```



As  $\lambda$  increases, the model becomes less flexible, and the variance decreases while the bias increases. Generally, the reduction in variance outweighs the increase in bias until the value of  $\lambda$  that minimizes the test error is reached, which represents a noteworthy trade-off.

6.

```
best_coefs <- coef(cvglmnet, s = "lambda.min")
best_coefs_dt <- data.table(word=rownames(best_coefs),
                           coef=best_coefs[,1])
best_coefs_dt[order(coef,decreasing = T)]
```

|       | word     | coef       |
|-------|----------|------------|
|       | <char>   | <num>      |
| 1:    | atlanta  | 0.1492730  |
| 2:    | Npme     | 0.1425327  |
| 3:    | patriot  | 0.1371343  |
| 4:    | colorado | 0.1347370  |
| 5:    | sacrific | 0.1294068  |
| ---   |          |            |
| 1492: | vulner   | -0.1316845 |
| 1493: | view     | -0.1323054 |

```
1494:      visit -0.1328930
1495:      volunt -0.1334961
1496: (Intercept) -0.1395247
```

After extracting the coefficients at  $\lambda_{\min}$ , I found that words with positive coefficients (such as atlanta, patriot, and colorado) are more likely to indicate a Trump tweet, while words with negative coefficients (such as vulner, volunt, and view) are more likely to indicate a Bernie tweet. This is generally consistent with my expectations regarding the language styles of the two politicians: Trump's tweets tend to focus on geographic locations or patriotic topics, while Bernie's tweets highlight progressive social themes, emphasizing social vulnerability and volunteer-related activities.

## Part 2: Social Network Ad Purchase

1.

```
sna_data <- fread("~/Documents/ML/Labs/Lab1/Kaggle_Social_Network_Ads.csv")
sna_data$Purchased <- factor(sna_data$Purchased, levels = c(0,1))
```

2.

```
# Use package "caret" to do k-fold cross-validation
tc <- caret::trainControl(method = 'cv', number = 5)

set.seed(12345) # Set to ensure that folds are the same across models
nonlinear_model <- caret::train(Purchased ~ .,
                                data = sna_data,
                                method = "glm",
                                trControl = tc,
                                family = "binomial")

nonlinear_model$results
```

|   | parameter | Accuracy  | Kappa     | AccuracySD | KappaSD   |
|---|-----------|-----------|-----------|------------|-----------|
| 1 | none      | 0.8452399 | 0.6500518 | 0.05814504 | 0.1348846 |

The test accuracy is 0.8452399.

3.

```
set.seed(12345) # Set to ensure that folds are the same across models
gam_model1 <- caret::train(Purchased ~ ns(Age,2) + ns(Salary, 2) + Gender,
                           data = sna_data,
                           method = "glm",
                           trControl = tc,
                           family = "binomial")
gam_model1$results
```

|   | parameter | Accuracy  | Kappa     | AccuracySD | KappaSD   |
|---|-----------|-----------|-----------|------------|-----------|
| 1 | none      | 0.8974621 | 0.7732889 | 0.03905348 | 0.0899952 |

```
gam_model2 <- caret::train(Purchased ~ ns(Age,3) + ns(Salary, 3) + Gender,
                           data = sna_data,
                           method = "glm",
                           trControl = tc,
                           family = "binomial")
gam_model2$results
```

|   | parameter | Accuracy  | Kappa     | AccuracySD | KappaSD    |
|---|-----------|-----------|-----------|------------|------------|
| 1 | none      | 0.9049305 | 0.7903603 | 0.02142775 | 0.04775425 |

```
gam_model3 <- caret::train(Purchased ~ ns(Age,4) + ns(Salary, 4) + Gender,
                           data = sna_data,
                           method = "glm",
                           trControl = tc,
                           family = "binomial")
gam_model3$results
```

|   | parameter | Accuracy  | Kappa     | AccuracySD | KappaSD    |
|---|-----------|-----------|-----------|------------|------------|
| 1 | none      | 0.9126758 | 0.8098498 | 0.02977598 | 0.06623301 |

a.

The test accuracy has improved, confirming the effectiveness of the updated model.

**b.**

This indicates that the previous standard logistic regression model was underfitting. In other words, the earlier linear model was insufficient to capture the patterns in this dataset, so we need a more flexible model to fit it—namely, the GAM. Compared to the linear model, the GAM introduces more bias.

**c.**

I would select GAM 3, as it achieves the highest test accuracy among these models.

**4.**

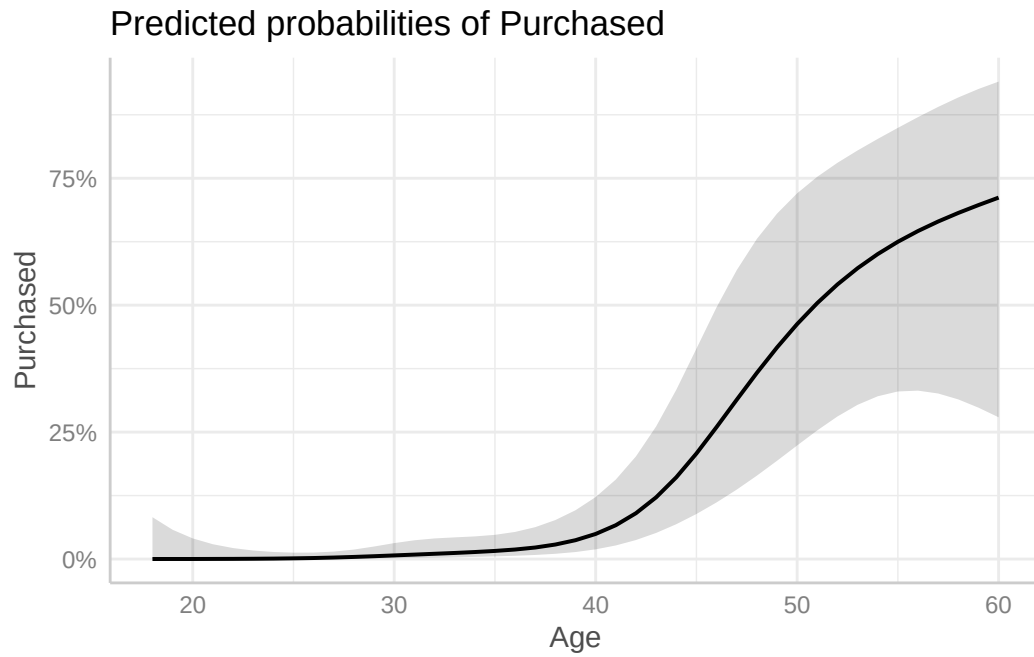
```
final_model <- glm(Purchased ~ ns(Age,4) + ns(Salary, 4) + Gender,  
                  data=sna_data,  
                  family = "binomial")  
ggpreds <- ggpredict(final_model)
```

Some of the focal terms are of type `character`. This may lead to unexpected results. It is recommended to convert these variables to factors before fitting the model.

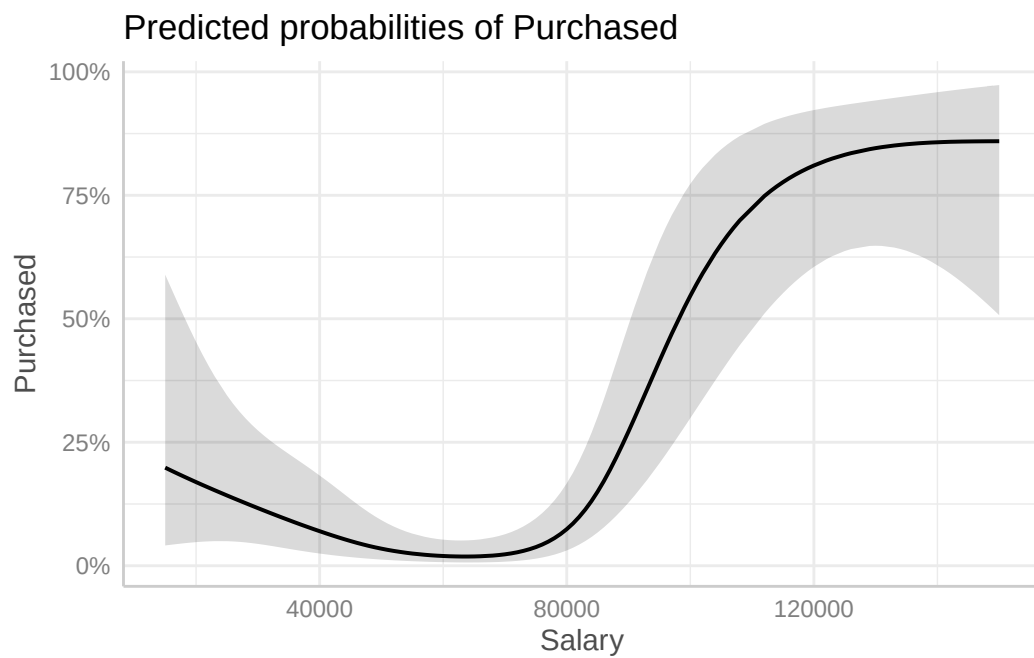
The following variables are of type character: `Gender`

```
plot(ggpreds)
```

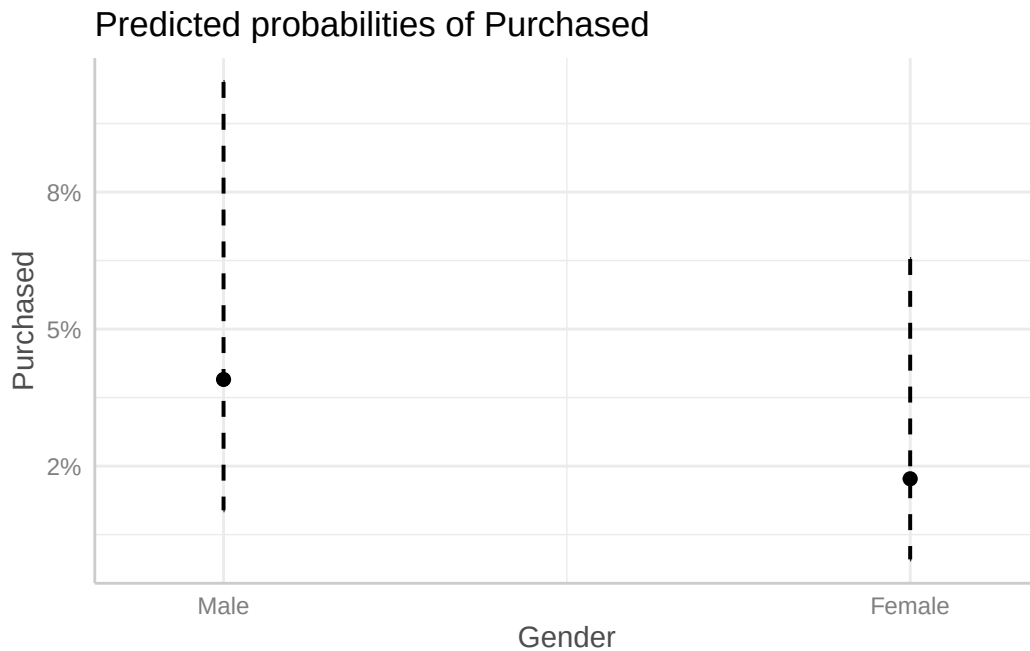
\$Age



\$Salary



\$Gender



The results show that the effects of Age and Salary on the probability of purchase are both non-linear, indicating that a standard linear logistic regression cannot fully capture these nonlinear effects.

5.

No, because Ridge and Lasso are intended to increase bias in order to mitigate overfitting. However, the current situation involves underfitting, which requires a reduction in bias.

### Part 3: K-fold cross validation from scratch (bonus!)

```
dt <- readRDS("~/Documents/ML/Labs/Lab1/dt.rds")  
head(dt)
```

| X     | Y     |
|-------|-------|
| <num> | <num> |

```

1: 2.2090390 0.2627579
2: 3.7577319 1.3698040
3: 2.6098233 0.5606494
4: 3.8612457 -0.3074609
5: -0.4351904 -1.0925596
6: -3.3362821 0.4714156

```

```

chunk2 <- function(x,n) split(x, cut(seq_along(x), n, labels = FALSE))

cv_poly <- function(k, d, dt) {
  n <- nrow(dt)
  folds <- chunk2(1:n, k)
  mse_list <- c()
  for (i in 1:k){
    test_idx <- folds[[i]]
    train_idx <- setdiff(1:n, test_idx)
    train_data <- dt[train_idx, ]
    test_data <- dt[test_idx, ]

    model <- lm(Y ~ poly(X, d), data = train_data)
    preds <- predict(model, newdata = test_data)

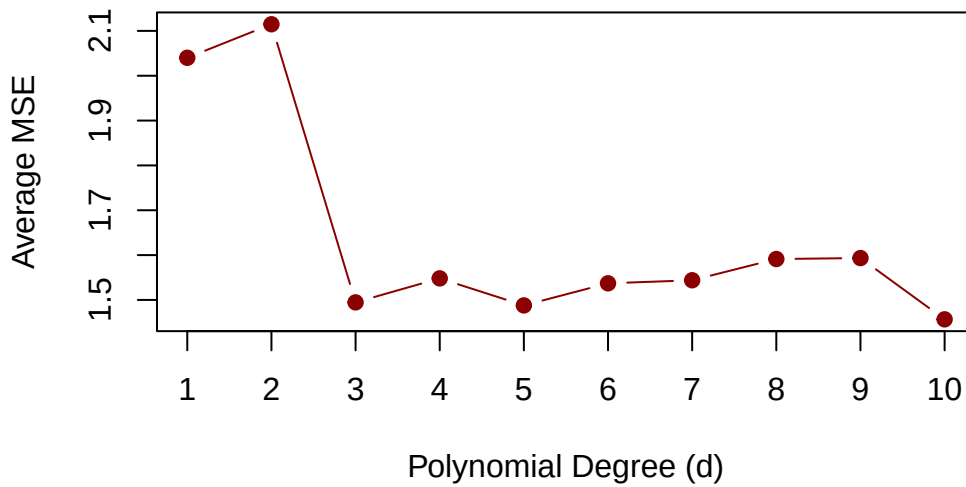
    mse <- mean((test_data$Y - preds)^2)
    mse_list <- c(mse_list, mse)
  }
  return(mean(mse_list))
}

d_values <- 1:10
mse_values <- sapply(d_values, function(d) cv_poly(5, d, dt))
plot(d_values, mse_values, type = "b",
     pch = 19, col = "darkred", lty = 1,
     xlab = "Polynomial Degree (d)",
     ylab = "Average MSE",
     main = "Average MSE Across Polynomial Degrees",
     xaxt = "n")

axis(1, at = d_values, labels = d_values)

```

### Average MSE Across Polynomial Degrees



The results show that as the polynomial degree  $d$  increases, the average MSE initially decreases and then rises, indicating that the model captures some nonlinearity in the data, but overly high-degree polynomials may lead to overfitting. The minimum average MSE is observed at  $d = 10$ ; however, this may reflect overfitting due to the small dataset. Choosing lower-degree polynomials such as  $d = 3$  or  $d = 5$  achieves comparably low average MSE while ensuring a more robust and interpretable model.